

Pratt Polynomials for Algorithmic Music

Arithmetic filters, timbre design,
and polyphonic MIDI synthesis

Orges Leka

Polynomial discovery and signal-processing concept: Orges Leka

Research, technical, and editorial assistant: ChatGPT 6.1 Sol

Abstract

This white paper develops an integer-indexed vocabulary for sound. A recursive family of polynomials assigns an exact algebraic object to every positive integer. Its central identity, $f_{mn} = f_m f_n$, turns multiplication of integers into composition of filters. The same family supplies normalized finite impulse response filters, stable continuous-time filters, damped resonators, and the spectral weights of a polyphonic MIDI synthesizer. These are distinct audio constructions with shared arithmetic structure.

The intended audience is composers, live coders, instrument designers, and developers working with algorithmic music. Definitions, a table of 24 examples, short proofs, and five reproducible figures connect the mathematics to musical controls. A working offline synthesizer has rendered six supplied MIDI files, covering 39,208 notes and approximately 30 minutes of stereo audio. The proposal is a compositional system and a practical DSP design method; perceptual quality and real-time performance remain questions for further evaluation.

The core musical operation

An integer names a filter. Prime factors name its reusable modules. Multiplying their labels cascades the modules; musical pitch can be set independently of that arithmetic timbre.

Contents

1	A vocabulary for sound design	2
2	Definition of the polynomial family	2
3	The algebra that makes the family useful	4
4	Realization I: coefficient FIR filters	5
5	Realization II: stable analog filter prototypes	6
6	Realization III: root-based resonators and sonification	9
7	Realization IV: a polyphonic MIDI instrument	10
8	A compositional language for live coding	12
9	Digital implementation and numerical practice	13
10	Prototype results and verification	14
11	Experiments for the algorithmic music community	15
A	The root bound behind analog stability	16
B	Reference construction code	16
C	Reproducing the white paper and the instrument	17
	Authorship and contribution	18
	References	18

How to read this paper

For musical ideas, begin with Sections 1 and 8. For implementation, use Sections 4, 5, 7, and 9. Appendix A supplies the root-location argument behind the stability guarantee.

1 A vocabulary for sound design

Algorithmic music often separates two questions: which events should occur, and what should those events sound like? Integer sequences can answer the first question through note, duration, or rhythm mappings. Here they also answer the second. An integer n selects a polynomial f_n , and a declared DSP mapping converts that polynomial into a filter or timbre.

The useful feature is the relationship between labels. The sound associated with 35 need not be an arbitrary preset unrelated to 5 and 7. Its underlying filter can be defined as the cascade of their filters. The label 25 means two copies of the 5-module, while 77 means a 7-module followed by an 11-module. Such arithmetic relationships can become audible transformations.

Construction	Polynomial-to-signal mapping	Musical role
Coefficient FIR	Normalized coefficients become delay taps	Short coloration, smoothing, comb effects
Shifted analog filter	Roots become stable poles after an affine shift	Subtractive filtering and transient design
Remapped resonator	Root real and imaginary parts control damping and frequency	Bells, drones, percussive modes, sonification
Harmonic MIDI voice	Filter response weights harmonics of a prescribed note	Playable pitched instruments and polyphony

Table 1: Four realizations of one polynomial family. Their frequency responses and time responses are different; each realization must be specified.

For an algorithmic composer, the main advantages are concrete:

- **Reproducibility.** The index and mapping parameters determine the coefficients and the resulting response without training or random sampling.
- **Compositional structure.** Integer multiplication has an exact counterpart in cascaded filters. Repeated prime factors create repeated modules.
- **Controlled complexity.** The polynomial degree grows on a logarithmic scale with n , giving a transparent filter-order budget.
- **Stable realizations.** The constructions below provide explicit stability conditions and meaningful frequency and decay controls.
- **Independent pitch.** A harmonic realization retains MIDI tuning while the arithmetic index controls spectral color.

Increasing n does not imply a monotone progression of brightness, resonance, or perceived complexity. Factorization matters more than numerical proximity. This makes the family attractive as a structured compositional language rather than as a replacement for a conventional cutoff knob.

2 Definition of the polynomial family

Let $\mathbb{N} = \{1, 2, 3, \dots\}$ and let $\mathcal{P}$ be the set of primes. For a prime p , $v_p(n)$ denotes the number of times p divides n . Thus

$$n = \prod_{p \in \mathcal{P}} p^{v_p(n)}, \quad (2.1)$$

with only finitely many nonzero exponents.

The family used in this paper is defined by

$$f_1(x) = 1, \quad f_2(x) = x, \quad (2.2)$$

$$f_p(x) = 1 + f_{p-1}(x) = 1 + \prod_{q|(p-1)} f_q(x)^{v_q(p-1)} \quad \text{for odd primes } p, \quad (2.3)$$

$$f_n(x) = \prod_{p|n} f_p(x)^{v_p(n)} \quad \text{for general } n. \quad (2.4)$$

The products in these formulas run over prime divisors. The empty product for $n = 1$ is 1. Recursion is well founded: the definition at an odd prime uses prime factors of $p - 1$, all smaller than p .

We call these *Pratt polynomials*, following their relationship to the recursive factorization of $p - 1$ in Pratt trees. This paper credits Orges Leka with the discovery of the polynomial family and the idea of applying it to signal processing. A mathematical treatment of the family is given in Leka's note [1].

For example, $f_7 = 1 + f_6 = 1 + f_2 f_3 = x^2 + x + 1$. For 35, the ordinary factorization $35 = 5 \cdot 7$ gives

$$f_{35}(x) = (x^2 + 1)(x^2 + x + 1) = x^4 + x^3 + 2x^2 + x + 1. \quad (2.5)$$

The variable x is initially a formal polynomial variable. A DSP realization later replaces it by a delay, a shifted Laplace variable, or a complex frequency coordinate. The integer index n and the variable x have separate roles.

2.1 A table of exact examples

Write

$$f_n(x) = \sum_{k=0}^{\alpha(n)} a_{n,k} x^k, \quad \alpha(n) = \deg f_n. \quad (2.6)$$

All coefficients in Table 2 are computed as exact integers. The value $f_n(1)$ is the sum of the coefficients and is used to normalize the FIR realization. The separate identity $f_n(2) = n$ normalizes the analog one.

Table 2: The first 20 polynomials and four composite examples.

n	Prime factors	$f_n(x)$	$\alpha(n)$	$f_n(1)$
1	1	1	0	1
2	2	x	1	1
3	3	$x + 1$	1	2
4	2^2	x^2	2	1
5	5	$x^2 + 1$	2	2
6	$2 \cdot 3$	$x^2 + x$	2	2
7	7	$x^2 + x + 1$	2	3
8	2^3	x^3	3	1
9	3^2	$x^2 + 2x + 1$	2	4
10	$2 \cdot 5$	$x^3 + x$	3	2
11	11	$x^3 + x + 1$	3	3
12	$2^2 \cdot 3$	$x^3 + x^2$	3	2
13	13	$x^3 + x^2 + 1$	3	3

n	Prime factors	$f_n(x)$	$\alpha(n)$	$f_n(1)$
14	$2 \cdot 7$	$x^3 + x^2 + x$	3	3
15	$3 \cdot 5$	$x^3 + x^2 + x + 1$	3	4
16	2^4	x^4	4	1
17	17	$x^4 + 1$	4	2
18	$2 \cdot 3^2$	$x^3 + 2x^2 + x$	3	4
19	19	$x^3 + 2x^2 + x + 1$	3	5
20	$2^2 \cdot 5$	$x^4 + x^2$	4	2
25	5^2	$x^4 + 2x^2 + 1$	4	4
35	$5 \cdot 7$	$x^4 + x^3 + 2x^2 + x + 1$	4	6
49	7^2	$x^4 + 2x^3 + 3x^2 + 2x + 1$	4	9
77	$7 \cdot 11$	$x^5 + x^4 + 2x^3 + 2x^2 + 2x + 1$	5	9

3 The algebra that makes the family useful

Proposition 3.1 (Exact arithmetic identities). *For all $m, n \in \mathbb{N}$, the following hold:*

$$f_{mn}(x) = f_m(x)f_n(x), \quad (3.1)$$

$$f_n(2) = n, \quad (3.2)$$

$$\alpha(mn) = \alpha(m) + \alpha(n). \quad (3.3)$$

Each f_n is monic and has nonnegative integer coefficients.

Proof. The identity $v_p(mn) = v_p(m) + v_p(n)$ gives (3.1) directly from (2.4). Evaluation at 2 follows inductively: the base values are 1 and 2; for an odd prime, $f_p(2) = 1 + f_{p-1}(2) = p$; products then give the general case. Multiplication adds degrees. The recursion uses products and addition of a positive constant, which preserve integer nonnegativity and monicity. $\square$

The index is recoverable from the polynomial by evaluation at 2, so distinct integers have distinct polynomials. This injectivity is algebraic; it does not guarantee perceptually distinct sounds after bandwidth limits, normalization, envelopes, and playback.

The family preserves multiplication, but generally not ordinary addition. For example, $f_{2+3} = x^2 + 1$, whereas $f_2 + f_3 = 2x + 1$. Adding two audio signals therefore has no automatic interpretation as adding their integer labels.

Proposition 3.2 (Order grows logarithmically). *For every $n \geq 1$,*

$$2^{\alpha(n)} \leq n \leq 3^{\alpha(n)}, \quad \frac{\log n}{\log 3} \leq \alpha(n) \leq \frac{\log n}{\log 2}. \quad (3.4)$$

Proof. The lower bound follows from the leading term of the monic polynomial at 2 and its nonnegative coefficients. For the upper bound, induct on n . Products preserve it. If p is odd, set $d = \alpha(p-1) = \alpha(p)$. Induction gives $p-1 \leq 3^d$. Since $p-1$ is even and 3^d is odd, $p-1 \leq 3^d - 1$, hence $p \leq 3^d$. The bases 1 and 2 are immediate. $\square$

An order-7 polynomial already describes the MIDI piano index 350. The degree bound controls the number of coefficients or poles, not the computational difficulty of factoring an arbitrarily large integer. Practical instruments can use a precomputed range of modest indices.

The prime-index polynomials are irreducible over $\mathbb{Q}$ [1]. This supports their interpretation as canonical arithmetic building blocks. Their acoustic usefulness comes from the explicit filter mappings below.

4 Realization I: coefficient FIR filters

A finite impulse response (FIR) filter is the simplest direct implementation. Let $u[j]$ be an input signal indexed by sample number j . Define

$$c_n[k] = \frac{a_{n,k}}{f_n(1)}, \quad y[j] = \sum_{k=0}^{\alpha(n)} c_n[k] u[j-k]. \quad (4.1)$$

Since $f_n(1) > 0$, these taps are well defined, nonnegative, and sum to one. Their transfer function is

$$C_n(z) = \frac{f_n(z^{-1})}{f_n(1)}. \quad (4.2)$$

Here z^{-1} means a one-sample delay. Standard filter terminology and convolution conventions are described by Smith [2].

The following properties require no root calculation:

$$C_n(1) = 1, \quad (4.3)$$

$$|C_n(e^{i\varpi})| \leq \sum_k c_n[k] = 1, \quad (4.4)$$

$$C_{mn}(z) = C_m(z)C_n(z). \quad (4.5)$$

The first equation preserves a constant signal. The second follows from the triangle inequality and bounds the frequency-response magnitude. Finite taps also make the filter causal and stable. The third follows because both the numerator and its normalization are multiplicative.

Some small indices give familiar effects:

$$C_2(z) = z^{-1}, \quad C_3(z) = \frac{1}{2}(1 + z^{-1}), \quad (4.6)$$

$$C_5(z) = \frac{1}{2}(1 + z^{-2}), \quad C_7(z) = \frac{1}{3}(1 + z^{-1} + z^{-2}), \quad (4.7)$$

$$C_{35}(z) = \frac{1}{6}(1 + z^{-1} + 2z^{-2} + z^{-3} + z^{-4}). \quad (4.8)$$

Thus 2 is a pure delay, 3 is a two-tap average, 5 is a short comb, and 7 is a three-tap average. Multiplication by a power of 2 adds delay. The symmetric examples have linear phase, but the full family does not: f_{11} has coefficients $(1, 1, 0, 1)$.

For a more pronounced musical effect, replace the single-sample delay by an L -sample delay:

$$C_{n,L}(z) = \frac{f_n(z^{-L})}{f_n(1)}, \quad L \in \mathbb{N}. \quad (4.9)$$

For instance, $C_{5,L} = (1 + z^{-2L})/2$ has comb-notch spacing set by L and the sample rate. The cascade identity remains exact when every module uses the same L . As a groove or spectral-color control, L is often more audible than the integer index alone.

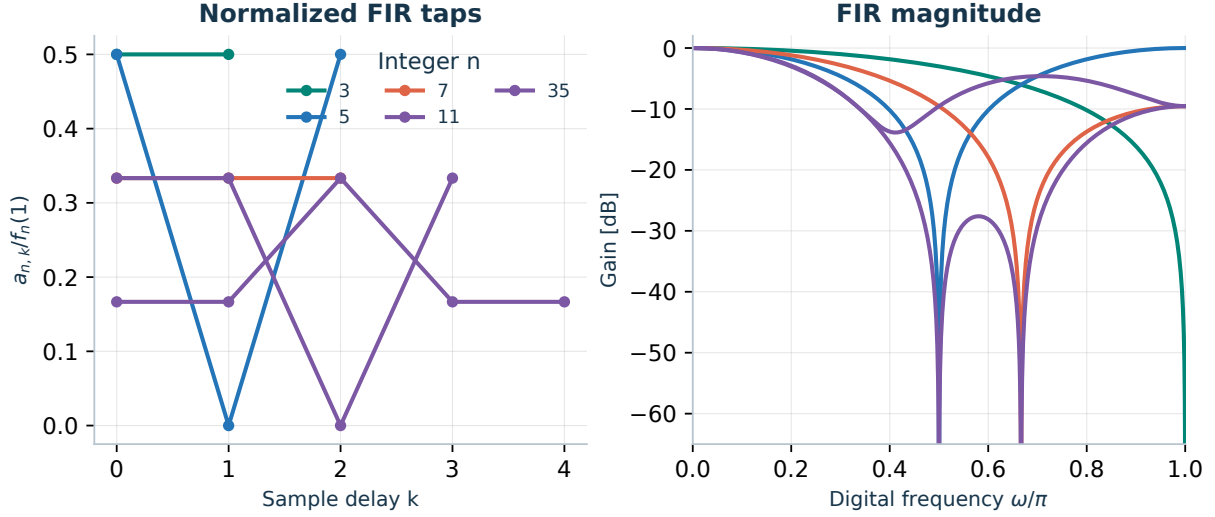


Figure 1: FIR taps and responses for five indices. The horizontal frequency axis is in units of the Nyquist frequency. Deep notches are genuine cancellations; the display floor is -65 dB. All five filters have unit DC gain.

5 Realization II: stable analog filter prototypes

Let $s \in \mathbb{C}$ be the Laplace variable and choose a common frequency scale $\omega_0 > 0$, measured in inverse seconds. Define

$$H_n(s) = \frac{n}{f_n(2 + s/\omega_0)}. \quad (5.1)$$

This is an all-pole rational filter. For $n > 1$ it is strictly proper, so its high-frequency response decays. For $n = 1$ it is the identity $H_1 = 1$. The identity $f_n(2) = n$ gives $H_n(0) = 1$.

If $\theta_1, \dots, \theta_d$ are the roots of f_n , counted with multiplicity, then

$$H_n(s) = \prod_{r=1}^d \frac{2 - \theta_r}{2 + s/\omega_0 - \theta_r}, \quad s_r = \omega_0(\theta_r - 2). \quad (5.2)$$

The root-location property $\operatorname{Re} \theta_r < 3/2$, proved in Appendix A, implies

$$\operatorname{Re} s_r < -\omega_0/2. \quad (5.3)$$

Every pole is strictly in the left half-plane. For each fixed integer index, the causal impulse response is absolutely integrable, so the filter is bounded-input, bounded-output stable. Repeated poles add polynomial factors in time, but do not change that conclusion.

n	$H_n(s)$ for $\omega_0 = 1 \text{ s}^{-1}$	Musical starting point
2	$2/(s + 2)$	One-pole smoothing
3	$3/(s + 3)$	Faster one-pole smoothing
5	$5/(s^2 + 4s + 5)$	Damped oscillatory transient
7	$7/(s^2 + 5s + 7)$	Another second-order color
11	$11/(s^3 + 6s^2 + 13s + 11)$	Third-order spectral contour
17	$17/(s^4 + 8s^3 + 24s^2 + 32s + 17)$	Two conjugate pole pairs

Table 3: Small analog prototypes. Audio-frequency operation requires an appropriate ω_0 ; these normalized numbers describe response shape.

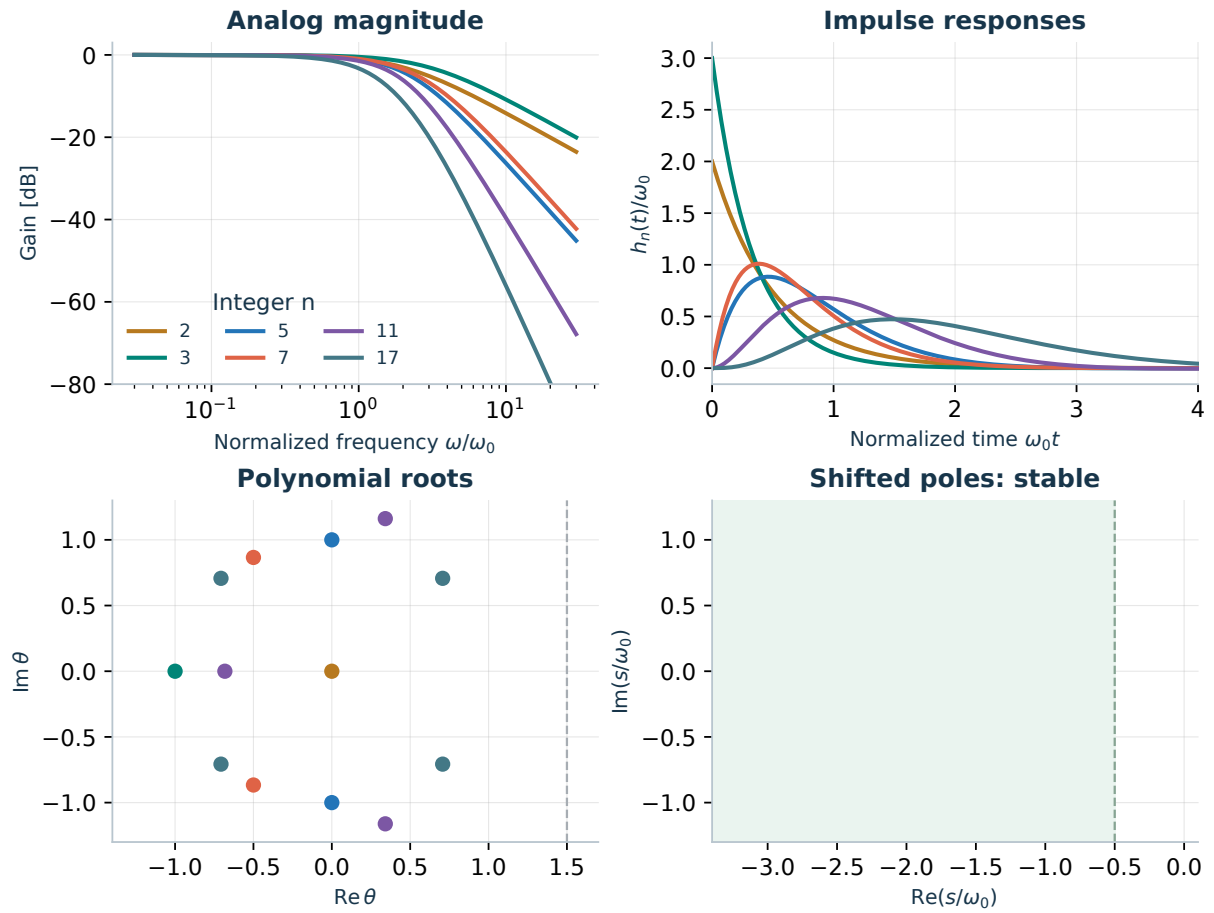


Figure 2: Magnitude, impulse responses, polynomial roots, and shifted filter poles. The dashed root boundary is $\text{Re } \theta = 1.5$; the pole boundary is $\text{Re}(s/\omega_0) = -0.5$. The responses use the same scale and are not individually peak normalized.

Unit DC gain is a normalization, not a guarantee of unit peak gain. Stability does not force a monotone low-pass response either. The asymptotic roll-off is $-20\alpha(n)$ dB per decade, but pole placement determines the details at finite frequencies.

5.1 Multiplication becomes cascading

For a shared scale ω_0 , substitution into (3.1) gives

$$H_{mn}(s) = H_m(s)H_n(s), \quad H_n(s) = \prod_{p|n} H_p(s)^{v_p(n)}. \quad (5.4)$$

If $h_n(t)$ denotes the causal impulse response for $n > 1$, then

$$h_{mn} = h_m * h_n, \quad (h_m * h_n)(t) = \int_0^t h_m(t-u)h_n(u) \, du. \quad (5.5)$$

Consequently, log magnitudes add:

$$20 \log_{10} |H_{mn}(i\omega)| = 20 \log_{10} |H_m(i\omega)| + 20 \log_{10} |H_n(i\omega)|. \quad (5.6)$$

This is a statement about passing a signal through two filters in sequence. It is different from mixing two independently generated sounds.

The simplest oscillatory example is

$$h_5(t) = 5\omega_0 e^{-2\omega_0 t} \sin(\omega_0 t), \quad t \geq 0. \quad (5.7)$$

Its integral is 1, matching DC gain. The repeated-factor index 25 squares the transfer function and convolves this transient with itself. Repetition thus changes the time profile as well as the attenuation slope.

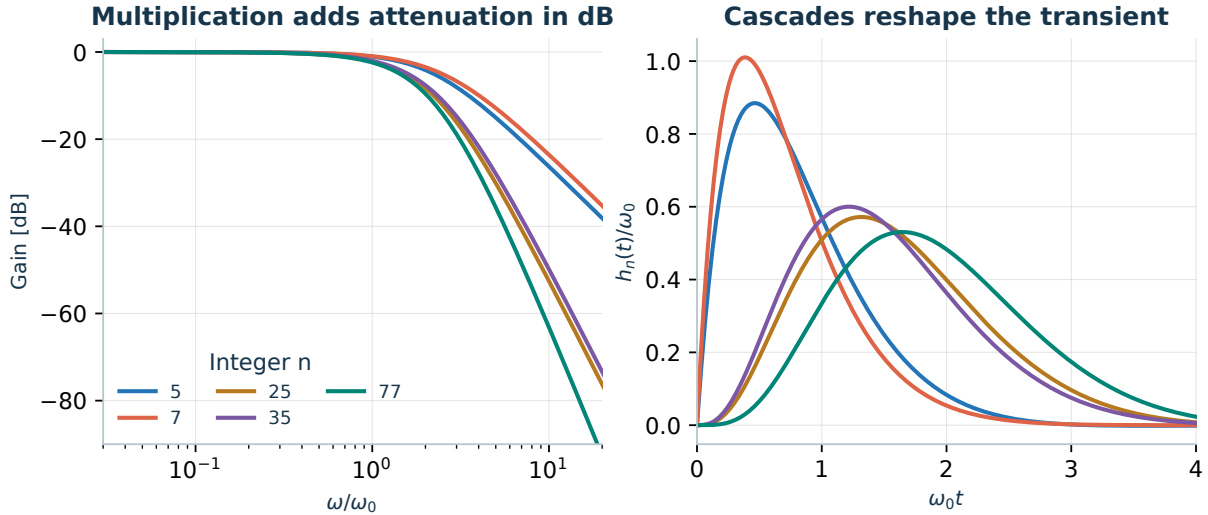


Figure 3: Arithmetic composition: $25 = 5^2$, $35 = 5 \cdot 7$, and $77 = 7 \cdot 11$. Their stronger roll-off and changed transients result from exact cascades at a shared scale. No individual gain adjustment is used.

6 Realization III: root-based resonators and sonification

The affine filter in (5.1) ties oscillation frequency and decay to one scale. For sonification, it is often useful to separate them. If $\theta_r = a_r + ib_r$, choose two positive global parameters γ and Ω , both in inverse seconds, and define

$$\lambda_r = -\gamma(2 - a_r) + i\Omega b_r, \quad (6.1)$$

$$\tilde{H}_n(s) = \prod_{r=1}^{\alpha(n)} \frac{-\lambda_r}{s - \lambda_r}. \quad (6.2)$$

The frequency and amplitude-decay time constant of a conjugate pole pair are

$$\nu_r = \frac{\Omega|b_r|}{2\pi} \text{ Hz}, \quad \tau_r = \frac{1}{\gamma(2 - a_r)} \text{ seconds}. \quad (6.3)$$

Increasing Ω transposes the modal frequencies. Increasing γ shortens their decays. Real roots generate nonoscillating decays.

Conjugate roots remain conjugate poles, so the filter has real coefficients. The stability bound gives $\text{Re } \lambda_r < -\gamma/2$, and every factor has DC gain 1. Since the root multiset of a product is the union of the two root multisets, including multiplicities,

$$\tilde{H}_{mn} = \tilde{H}_m \tilde{H}_n. \quad (6.4)$$

This identity requires the same γ and Ω for all compared filters. At $\gamma = \Omega = \omega_0$, the remapped family coincides with (5.1).

For $n = 5$, the roots are $\pm i$, so

$$\tilde{H}_5(s) = \frac{4\gamma^2 + \Omega^2}{(s + 2\gamma)^2 + \Omega^2}. \quad (6.5)$$

With $\gamma = 0.5 \text{ s}^{-1}$ and $\Omega = 2\pi \cdot 440 \text{ s}^{-1}$, the oscillation is 440 Hz and its amplitude time constant is 1 second. For $n = 7$, the roots are $-1/2 \pm i\sqrt{3}/2$, giving about 381.05 Hz and 0.8 seconds under the same mapping.

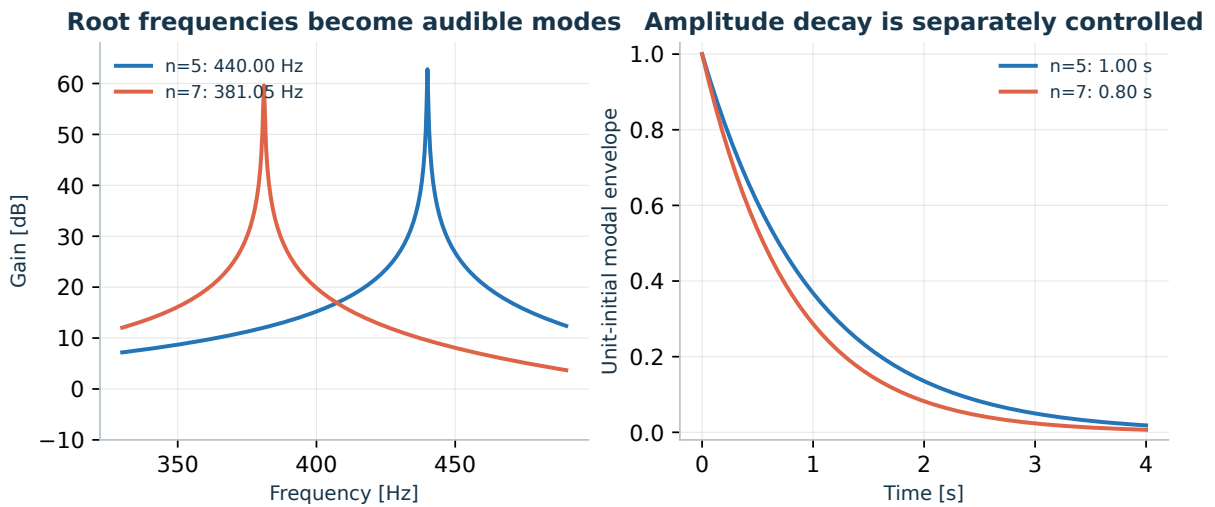


Figure 4: Two audible modal examples under one shared mapping. Left: actual DC-normalized filter magnitudes, which can have large resonant peaks. Right: unit-initial amplitude envelopes of the pole pairs, rather than normalized impulse-response amplitudes.

An impulse, a short noise burst, or an external recording can excite these filters. A separate additive modal instrument can instead sum damped oscillators at the same poles. The additive instrument and the product filter share modal locations but generally have different gains and numerators; only the declared product realization has (6.4).

This route gives the integer sequence its own pitches. It is appropriate for number sonification, but generally changes the pitches of a pre-existing score. A MIDI instrument needs the alternative in the next section.

7 Realization IV: a polyphonic MIDI instrument

For MIDI note number $m \in \{0, \dots, 127\}$, retain the prescribed equal-tempered fundamental

$$f_0(m) = 440 \cdot 2^{(m-69)/12} \text{ Hz.} \quad (7.1)$$

Let g denote an instrument preset and let its positive integer base be b_g . The prototype selects the arithmetic index

$$n_{m,g} = (m + 1)b_g. \quad (7.2)$$

For harmonic number k , sample the analog filter response at a normalized frequency determined by a positive brightness parameter ξ_g :

$$R_{m,g}(k) = \frac{n_{m,g}}{f_{n_{m,g}}(2 + i\xi_g k)}. \quad (7.3)$$

This is $H_{n_{m,g}}(2\pi i k f_0)$ when $\omega_0 = 2\pi f_0/\xi_g$. The filter follows the note's pitch, while its normalized spectral contour follows the polynomial.

Choose real excitation weights $A_{k,g} \geq 0$, typically of the form $k^{-\sigma_g}$. A sine-rich periodic waveform before gain normalization is

$$w_{m,g}(t) = \text{Re} \sum_{k=1}^{K_m} (-i A_{k,g} R_{m,g}(k)) e^{2\pi i k f_0(m)t}. \quad (7.4)$$

Both the magnitude and phase of the filter response are used. All harmonics remain integer multiples of f_0 ; the polynomial colors the sound without replacing MIDI tuning by root frequencies.

The existing renderer computes a 4,096-sample periodic wavetable with an inverse FFT, peak normalizes it, and interpolates it for each active voice. It retains at most 56 harmonics, with the additional bound

$$K_m \leq \left\lfloor \frac{0.42 F_s}{f_0(m)} \right\rfloor, \quad F_s = 44,100 \text{ Hz.} \quad (7.5)$$

The 0.42 factor leaves margin below Nyquist. This controls the stationary harmonic content; interpolation, rapid modulation, and envelopes can still produce small residual spectral artifacts and require separate measurement.

7.1 Voices, performance controls, and summation

Let a MIDI note event r have onset t_r , pitch m_r , velocity v_r , preset g_r , and envelope E_r . A simplified stereo rendering is

$$y_c(t) = \sum_r G_{r,c}(t) E_r(t - t_r) w_{m_r, g_r}(t - t_r), \quad c \in \{L, R\}, \quad (7.6)$$

Preset	b_g	Nominal ξ_g	σ_g	Envelope character
Piano	5	0.150	0.92	Fast attack, natural decay
Electric piano	11	0.135	0.90	Softer attack, longer decay
Organ	2	0.130	1.05	Held note, short release
Pluck / pizzicato	17	0.120	0.78	Short, bright attack
Strings	7	0.115	0.72	Held note, light vibrato
Brass	11	0.110	0.66	Richer excitation spectrum
Reed	5	0.110	0.80	Odd-harmonic emphasis
Flute	3	0.150	1.75	Softer upper harmonics
Pad	37	0.100	0.82	Slower attack and release

Table 4: Selected prototype settings. Preset names describe synthesis roles and envelope behavior, rather than measured emulations of acoustic instruments. Velocity modifies the nominal frequency coordinate and voice amplitude.

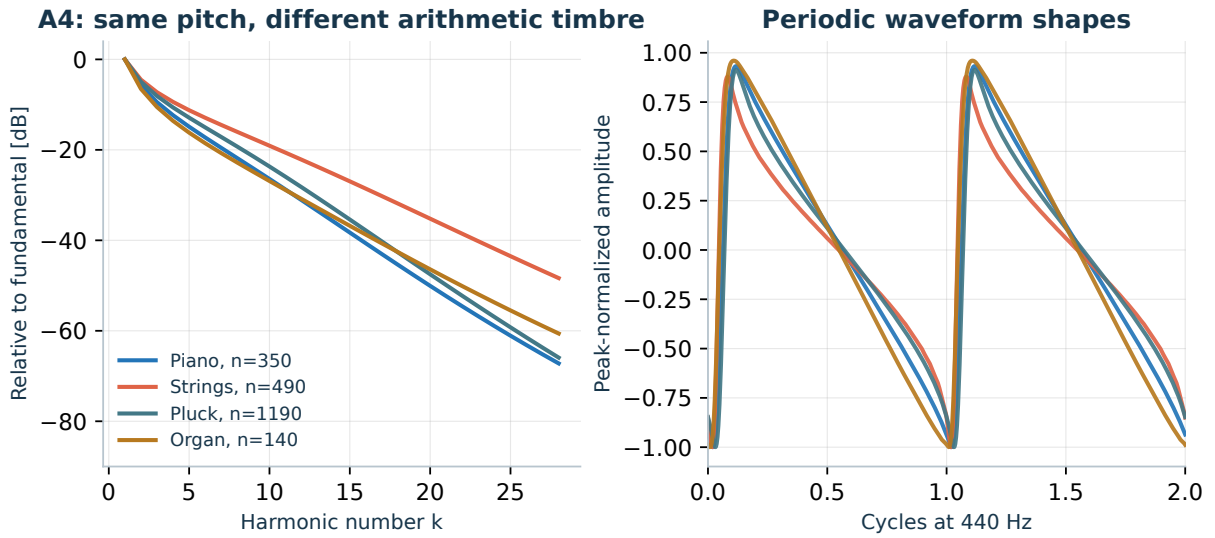


Figure 5: Four arithmetic timbres at A4, with the first 28 harmonics displayed. The indices are 350, 490, 1190, and 140. Settings use the prototype’s velocity bucket for MIDI velocity 96. Left: harmonic magnitude relative to the fundamental. Right: peak-normalized bare periodic waveforms, without envelopes or vibrato.

where $G_{r,c}$ contains velocity, channel volume, expression, and pan. Pitch bend and vibrato alter oscillator phase. Sustain changes note-release timing; the MIDI tempo map determines event times in seconds. A common room effect and a final linear gain adjustment follow the voice sum.

The renderer has no fixed voice-count limit. It supports MIDI formats 0 and 1, multiple tracks, tempo changes, running status, sustain, program changes, volume, expression, pan, and pitch bend. Percussion uses simple procedural voices. It is an offline command-line renderer, not a real-time plug-in.

The exact cascade law applies to the raw response under shared mapping parameters. Independent wavetable normalization, per-note frequency scaling, voice envelopes, summation, and effects are additional synthesis operations. The complete mixed music is not asserted to obey an integer-multiplication law. Likewise, the wavetable evaluates the steady-state harmonic filter response; its ADSR-style transitions do not simulate every analog filter transient.

8 A compositional language for live coding

8.1 Prime modules as a timbre score

Choose a finite collection of primes $p_1, \dots, p_J$. At step r , choose nonnegative exponents $e_{r,j}$ and form

$$n_r = \prod_{j=1}^J p_j^{e_{r,j}}. \quad (8.1)$$

For one shared analog mapping,

$$H_{n_r}(s) = \prod_{j=1}^J H_{p_j}(s)^{e_{r,j}}, \quad \alpha(n_r) = \sum_{j=1}^J e_{r,j} \alpha(p_j). \quad (8.2)$$

The exponents are a compositional score for the filter network. They say which modules are present, how many times each is repeated, and the total order.

Index	Cascade	Order	Suggested compositional gesture
5	H_5	2	Establish the initial color
7	H_7	2	Change color at the same order
35	$H_5 H_7$	4	Combine both colors through a cascade
25	H_5^2	4	Repeat one module rather than adding a new one
49	H_7^2	4	Compare two repeated-module transients
77	$H_7 H_{11}$	5	Add a third-order module

Table 5: A short arithmetic timbre phrase. These are design suggestions; the identity specifies the filtering operation, not a perceptual verdict.

A rhythm may drive the exponent updates while melody is generated separately. Changing one exponent creates a controlled change in module structure. The operation $n \mapsto np$ adds one p -module. Division by p , when $p \mid n$, removes it from the declared topology; it does not require an audio inverse filter, which may be unsuitable or noncausal.

8.2 Integer sequences as timbre modulation

An integer sequence $n_0, n_1, \dots$ can select responses at successive note events. A source sequence, including a user-selected sequence from a database, does not by itself specify the music: mapping choices determine note pitch, event spacing, gain, and timbre.

For large sequence values, map into a curated bank $\mathcal{B}$ using a declared index-selection rule. A modulo mapping is inexpensive but discards factorization relationships. Selecting prime exponents directly, as in (8.1), preserves those relationships.

One useful constraint is a filter-order budget D :

$$\sum_j e_{r,j} \alpha(p_j) \leq D. \quad (8.3)$$

This supports predictable real-time resource allocation. An application can offer the composer three separate controls: arithmetic structure, overall frequency scale, and amplitude or decay. Keeping them separate makes the instrument easier to explore.

8.3 Continuous transitions between integer labels

Integer selection is discrete, but playback transitions need not be abrupt. For two fixed stable filters driven by the same signal u , let

$$y(t) = (1 - \eta(t)) (h_a * u)(t) + \eta(t) (h_b * u)(t), \quad 0 \leq \eta(t) \leq 1. \quad (8.4)$$

This is a smooth output crossfade when η is smooth. Bounded weights and stable branches make the result bounded for bounded input. The intermediate response generally is not another member of the integer-indexed family.

Another proposed control is a fractional response. On a chosen frequency interval with a continuous phase convention, define

$$H_\eta(i\omega) = \exp \left(\sum_j \eta_j \text{Log } H_{p_j}(i\omega) \right), \quad \eta_j \geq 0. \quad (8.5)$$

At integer weights it reproduces the corresponding cascade on that interval. For noninteger weights, branch choice, causality, and implementation need separate treatment; this spectral interpolation is a research proposal rather than an implemented filter guarantee. Output crossfading is the simpler production method.

8.4 Polynomial waveshaping as an additional experiment

For $n > 1$ and drive $g > 0$, a local unit-slope waveshaper is

$$W_{n,g}(u) = \frac{f_n(2 + gu) - n}{gf'_n(2)}. \quad (8.6)$$

It satisfies $W_{n,g}(0) = 0$ and $W'_{n,g}(0) = 1$. For instance,

$$W_{5,g}(u) = u + \frac{g}{4}u^2, \quad W_{7,g}(u) = u + \frac{g}{5}u^2. \quad (8.7)$$

These maps add level-dependent harmonics and generally produce DC offsets. They are unbounded polynomials, so an instrument must define a bounded input range, gain staging, DC removal, and oversampling with proper resampling filters. Their arithmetic identity is not the filter-cascade identity. This route has not been used in the six MIDI renders reported below.

9 Digital implementation and numerical practice

9.1 Three implementation strategies

FIR coefficients. Compute exact $a_{n,k}$, normalize by $f_n(1)$, and convolve with the input. This has finite memory and needs no numerical roots. Sparse coefficients allow inexpensive implementations of small indices.

IIR sections. Factor the denominator into real first- and second-order sections. Conjugate roots belong in the same second-order section. Cascading small sections with controlled internal gains is generally preferable to expanding a high-order denominator. Repeated prime factors can reuse the same section coefficients with independent state.

Wavetable voices. Compute the complex response at harmonic positions, build the table once, and cache it by pitch, preset, and velocity bucket. Playback then needs phase accumulation and table interpolation. This is the strategy used by the supplied MIDI prototype.

9.2 From analog prototypes to digital filters

The bilinear transform is one standard choice [3]:

$$s = 2F_s \frac{1 - z^{-1}}{1 + z^{-1}}, \quad H_n^{\text{dig}}(z) = H_n \left(2F_s \frac{1 - z^{-1}}{1 + z^{-1}} \right). \quad (9.1)$$

It maps left-half-plane poles inside the unit circle. Substituting the same transform for each family member preserves the cascade identity. The analog and digital frequency coordinates satisfy

$$\omega = 2F_s \tan(\varpi/2), \quad (9.2)$$

so frequency warping must be considered when choosing an audio cutoff or a modal frequency. A shared prewarping convention preserves composition; individually retuning modules changes which responses are being compared.

For remapped modes, a separate discrete pole construction is

$$r_j = e^{\lambda_j/F_s}, \quad D_n(z) = \prod_j \frac{1 - r_j}{1 - r_j z^{-1}}. \quad (9.3)$$

Since $|r_j| < 1$, this is stable and has $D_n(1) = 1$. It preserves composition when the pole mapping and sample rate are shared. It matches pole locations, not the complete analog frequency response. Frequencies must be restricted before sampling to avoid aliasing of modal angles.

9.3 Exact arithmetic and finite precision

Polynomial construction should use arbitrary-precision integer coefficients. Horner evaluation is appropriate for frequency responses. For large orders or indices, prime-factor or section-based evaluation can reduce coefficient growth and numerical sensitivity.

Root finding introduces numerical error, especially at repeated roots. Constructing a composite filter from known prime modules avoids recomputing nearly repeated roots from a fully expanded polynomial. Mathematical pole stability also does not bound the internal gain of every numerical realization. Gain staging and finite-precision checks remain part of implementation.

The polynomial degree bound does not imply cheap factorization of huge indices. Precompute indices, factorization, coefficients, and section data outside an audio callback. Coefficient or preset changes should be smoothed or crossfaded. Stability of every frozen coefficient set alone is insufficient to guarantee arbitrary rapidly time-varying IIR behavior.

10 Prototype results and verification

The working implementation rendered the six supplied MIDI performances in Table 6. Each output is stereo MP3 at 44.1 kHz and 192 kbit/s, with 3.2 seconds added for release and room decay. Original MIDI note timing and tempo maps were retained.

The collection contains 39,208 notes and 1,809.15 seconds of audio. The largest active-note count is 100 in the Moonlight performance, where sustain extends many overlapping notes. This verifies a useful polyphonic workload for the offline renderer; it is not a measured real-time latency benchmark.

Validation covered the following:

- Exact coefficient identities and $f_n(2) = n$, recomputed in the white-paper assets for indices 1 through 1,000.

MIDI performance	Duration	Notes	Peak polyphony	Tempos
Vivaldi: Summer, III	2:47	7,918	16	15
Vivaldi: Spring, III	4:35	5,414	15	66
Satie: Gymnopédie No. 1	3:58	455	13	1
Beethoven: Moonlight, III	7:58	6,550	100	1
Vivaldi: Summer, III (D4 file)	2:47	7,918	21	15
Beethoven: Symphony No. 5, I	8:04	10,953	22	122

Table 6: Completed offline renders. Peak polyphony counts simultaneously active MIDI note instances, including sustain extension; it is not a count of distinct pitches. Durations are rounded to the nearest second. The D4 entry preserves the supplied variant’s filename label.

- The logarithmic degree bounds over the same range.
- An analytical check of $h_5(t)$ and a numerical check of $H_{35} = H_5 H_7$.
- A constructed MIDI test with known tempo-change, running-status, and sustain timings, and an A4 tuning test.
- Complete decode of every rendered MP3 to check duration, sample count, finite samples, and peaks below clipping. The final decoded peak range is approximately 0.747–0.763 in normalized floating-point audio.
- Conservation of input note counts in the rendering reports.

The five figures are numerical illustrations, not proofs of universal behavior. The algebraic statements and stability argument establish the general results. The prototypes demonstrate functionality; no listening panel, perceptual distinctness study, or comparison against commercial instruments has yet been performed. Modulation aliasing and real-time performance also remain to be measured in a dedicated implementation.

11 Experiments for the algorithmic music community

The next studies can be both musically useful and reproducible.

An arithmetic timbre atlas. Render a fixed test excitation through a bank of indices at shared settings. Publish coefficients, responses, and loudness-matched audition files. Compare prime, squarefree, and repeated-factor indices at matched filter order so that label structure can be assessed apart from order alone.

A live-coding instrument. Expose the prime-exponent vector as the score, with separate scale, damping, drive, and gain controls. Implement cached FIR or second-order sections and measure worst-case update time and CPU load. The arithmetic controls could be integrated into environments such as SuperCollider, Pure Data, Max, or a Faust-based processor; these integrations are proposed rather than delivered by this prototype.

A MIDI timbre model. Compare the current note-dependent index $(m+1)b_g$ with a fixed per-instrument index. The first gives arithmetic color variation across the keyboard; the second gives a more uniform normalized spectral identity. Keep envelopes and gains fixed during comparison.

A perceptual study. Evaluate whether listeners recognize factor-based relationships after normalization. Distinct polynomials and exact cascade laws do not by themselves establish distinguishable timbres or musically preferred relationships. Loudness, pitch, excitation, and temporal envelope should be controlled in such a study.

A sequence-to-sound workbench. Let composers declare separate mappings for pitch, event

timing, and timbre index, then retain those mappings as a score. This makes a sequence's sonification inspectable and repeatable without suggesting that one mapping is its uniquely correct musical interpretation.

Practical conclusion

The useful result is an arithmetic grammar for DSP: labels are reproducible, factorization specifies modules, and shared mappings turn multiplication into cascading. Musical pitch, performance controls, and perceptual evaluation complete the instrument.

A The root bound behind analog stability

Here is the root-location argument underlying the filter mapping [1], with the base case $p = 2$ treated separately.

For $\operatorname{Re} z \geq 3/2$, $|z| \geq 3/2$. The odd prime bases satisfy

$$|f_3(z)| = |z + 1| \geq 5/2, \quad |f_5(z)| = |z^2 + 1| = |z - i||z + i| \geq (\operatorname{Re} z)^2 \geq 9/4. \quad (\text{A.1})$$

Induct on odd primes. For $p \geq 7$ with an odd prime divisor q of $p - 1$, the product defining f_{p-1} contains z and $f_q(z)$, while all other factors have modulus at least 1. Thus

$$|f_p(z)| \geq |f_{p-1}(z)| - 1 > \frac{3}{2} \cdot 2 - 1 = 2. \quad (\text{A.2})$$

Otherwise $p - 1 = 2^r$ with $r \geq 3$, and

$$|f_p(z)| = |1 + z^r| \geq |z|^r - 1 \geq (3/2)^3 - 1 > 2. \quad (\text{A.3})$$

No odd-prime polynomial vanishes in this half-plane. The root of $f_2 = x$ is 0; composite roots come from prime factors. Therefore every root satisfies

$$\boxed{\operatorname{Re} \theta < 3/2.} \quad (\text{A.4})$$

The stronger modulus statement $|f_p(z)| > 2$ is for odd primes; it is not claimed for $f_2(z) = z$.

B Reference construction code

The coefficients below are stored in ascending order, so $(1, 0, 1)$ means $1 + x^2$. Multiplication uses exact integers. This compact implementation is intended for modest indices and can be replaced by a sieve and cached prime factorizations for a larger bank.

```
from functools import lru_cache

def factors(n):
    out, p = {}, 2
    while p*p <= n:
        while n % p == 0:
            out[p] = out.get(p, 0) + 1
            n //= p
        p += 1
    if n > 1:
        out[n] = out.get(n, 0) + 1
    return out
```

```
def multiply(a, b):
    out = [0] * (len(a) + len(b) - 1)
    for i, ai in enumerate(a):
        for j, bj in enumerate(b):
            out[i+j] += ai*bj
    return tuple(out)

@lru_cache(None)
def pratt(n):
    if n < 1:
        raise ValueError("Use a positive integer")
    if n == 1:
        return (1,)
    if n == 2:
        return (0, 1)
    fac = factors(n)
    if fac == {n: 1}:
        out = list(pratt(n-1))
        out[0] += 1
        return tuple(out)
    out = (1,)
    for p, exponent in fac.items():
        for _ in range(exponent):
            out = multiply(out, pratt(p))
    return out
```

For the steady-state MIDI spectrum, only a few more lines are needed:

```
import numpy as np

m, base, xi, roll = 69, 5, 0.15, 0.92
f0 = 440.0 * 2**((m-69)/12)
n = (m+1) * base
k = np.arange(1, 29)
R = n / np.polynomial.polynomial.polyval(
    2 + 1j*xi*k, pratt(n))
complex_harmonics = -1j * k**(-roll) * R
```

This short example uses nominal brightness. The full renderer adds velocity-dependent brightness, bandwidth limits, waveform normalization, performance controls, envelopes, and stereo mixing.

C Reproducing the white paper and the instrument

The companion source bundle contains this `.tex` document, generated tables, vector figures, the figure-generation script, CSV data, numerical checks, and the reference MIDI-renderer source. The six rendered MP3s were provided separately. Original MIDI files are required to reproduce those particular performances; their existing source credits should be retained.

With Python, NumPy, SciPy, Matplotlib, and a standard pdfLaTeX installation:

```
python generate_assets.py
pdflatex -interaction=nonstopmode \
    pratt_polynomials_algorithmic_music.tex
pdflatex -interaction=nonstopmode \
    pratt_polynomials_algorithmic_music.tex
```

Alternatively, `python build_pdf.py` regenerates figures and performs the LaTeX passes. The numerical input summary in `data/` is copied from the completed MIDI rendering reports, not inferred from audio.

For a new MIDI file, with FFmpeg and ffprobe on the executable search path:

```
python implementation/pratt_midi_synth.py \
    "input.mid" --output "output_pratt.mp3"
```

The bundled `validate_synth.py` checks the basic timing and tuning cases. The figure generator checks exact polynomial identities, degree bounds, the example cascade, and the analytical impulse response. Those finite checks supplement the proofs; they do not replace them.

Authorship and contribution

Author: Orges Leka. Discovery of the polynomial family; the idea of using it for signal processing, sonification, and algorithmic music; project direction and interpretation.

Assistant: ChatGPT 6.1 Sol. Assistance with DSP formulations, derivations, reference implementation, numerical checks, diagrams, and English white-paper preparation. The mathematical source [1] is authored by Orges Leka.

References

- [1] Orges Leka. *Galois Theory of the Polynomials $f_n(x)$, Pratt Trees, and Mersenne Primes*. 15 April 2026. See Definition 1.1, Proposition 1.2, Lemma 2.2, and Theorem 2.3. https://www.orges-leka.de/galois_pratt_theory.pdf.
- [2] Julius O. Smith III. *Introduction to Digital Filters with Audio Applications*. Online book, Center for Computer Research in Music and Acoustics, Stanford University. <https://www.dsprelated.com/freebooks/filters/>.
- [3] Julius O. Smith III. *Physical Audio Signal Processing*, section “Bilinear Transformation.” Online book, Center for Computer Research in Music and Acoustics, Stanford University. https://www.dsprelated.com/freebooks/pasp/Bilinear_Transformation.html.
- [4] Orges Leka, with assistance from ChatGPT 6.1 Sol. *Polyphonic Pratt MIDI Synthesizer*: Python reference implementation, six offline render reports, and validation scripts. Project artifacts, 6 October 2026.